

Woxsen University

School of Technology



B.Tech – Computer Science and Engineering

PROJECT BASED LEARNING REPORT

Title of the Project

SWIFTCOURIER: COURIER PARCEL BOOKING AND TRACKING MANAGEMENT SYSTEM

Course Code: 25TU03MJM0

Course Name: Database Management System

Semester: 3

Academic Year: 2026–27

Submitted by:

A Srestadeep – 25WU0101001

Abhigyan Gogoi – 25WU0101004

Jogi Dhanush – 25WU0101053

Section: CSE TIGERS

Under the Guidance of:

Faculty Name: Dr. Kiran Mayee Adavala

Designation: Professor

Department of Computer Science and Engineering

October, 2026

CERTIFICATE

This is to certify that the Project titled “**SwiftCourier: Courier Parcel Booking and Tracking Management System**” is a bonafide work carried out by A Srestadeep (25WU0101001), Abhigyan Gogoi (25WU0101004) and Jogi Dhanush (25WU0101053), B.Tech – Computer Science and Engineering, during the Academic Year 2026–27, in partial fulfillment of the requirements of the course Database Management System (25TU03MJM0), Semester 3, at the School of Technology, Woxsen University, under my supervision and guidance.

To the best of my knowledge, the work embodied in this report has not been submitted elsewhere for the award of any degree, diploma or other academic qualification.

Guide Signature

Dr. Kiran Mayee Adavala

Professor

Department of Computer Science and Engineering

School of Technology, Woxsen University

DECLARATION

We hereby declare that the Project titled “**SwiftCourier: Courier Parcel Booking and Tracking Management System**” submitted by us is an original work carried out under the guidance of Dr. Kiran Mayee Adavala, Professor, Department of Computer Science and Engineering, School of Technology, Woxsen University.

We confirm that this work has not been submitted in part or full for the award of any degree, diploma or other academic qualification in this or any other institution.

All sources of information used in this project have been properly acknowledged and cited as per university referencing standards. We further declare that the plagiarism percentage is within the permissible limit prescribed by the institution.

Place: _____

Date: _____

Signature of the Candidates

A Srestadeep
25WU0101001

Abhigyan Gogoi
25WU0101004

Jogi Dhanush
25WU0101053

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who helped us complete this project, SwiftCourier: Courier Parcel Booking and Tracking Management System.

First and foremost, we thank our guide, Dr. Kiran Mayee Adavala, Professor, Department of Computer Science and Engineering, for her guidance, encouragement and valuable feedback throughout the Database Management System course. Her emphasis on sound database design, normalisation and the correct use of constraints shaped every part of this work.

We are grateful to the Department of Computer Science and Engineering and the School of Technology, Woxsen University, for providing the facilities, software and learning environment needed to design, build and test the system.

We also thank the developers and maintainers of the free and open-source tools on which the project is built, including PHP, MariaDB, Apache, XAMPP, Bootstrap and the qrcode-generator library, which allowed the complete system to be developed at no cost.

Finally, we thank our classmates for their suggestions during testing, and our families for their constant support and encouragement.

A Srestadeep – 25WU0101001
Abhigyan Gogoi – 25WU0101004
Jogi Dhanush – 25WU0101053

ABSTRACT

Small and medium-sized courier operators still rely on paper registers, spreadsheets and telephone calls to book parcels and to answer the question every customer asks: “where is my parcel?” Records are duplicated, charges are worked out by hand, and when a status is overwritten the earlier steps of the journey are lost. This project applies relational database design to that problem.

SwiftCourier is a database-driven web application built with PHP 8, MySQL/MariaDB, HTML, CSS, Bootstrap 5 and vanilla JavaScript, running on XAMPP. Its database, *courier_management*, has four normalised tables (users, parcels, tracking_history and payments) and one view, linked by foreign keys and protected by primary-key, UNIQUE, NOT NULL, CHECK and ENUM constraints. Customers register and book parcels; the system calculates the delivery charge, generates a unique tracking ID inside a transaction, and produces a QR code that opens the parcel’s public tracking page. Administrators update status, location and remarks through an explicit state-transition rule, and every change is stored as a new tracking-history row in the same transaction, so the journey is never overwritten. Anyone can track a parcel without logging in, by tracking ID or by scanning the QR code with a phone. Dashboard statistics are computed with SQL aggregates and joins, and delete operations demonstrate ON DELETE CASCADE and RESTRICT.

The schema was analysed up to Boyce–Codd Normal Form. All 44 automated functional and access-control checks passed, six invalid statements were rejected by the database, 50 concurrent bookings received 50 distinct consecutive tracking IDs, and the QR code was scanned successfully with a smartphone. Queries completed in under 1 ms and pages in 31–44 ms (median).

Keywords: Relational Database, MySQL, Normalisation, BCNF, Transactions, Referential Integrity, Parcel Tracking, QR Code, PHP

TABLE OF CONTENTS

CERTIFICATE	ii
DECLARATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF FIGURES	vii
LIST OF TABLES	viii
CHAPTER 1: INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Scope and Organisation of the Report	2
CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES	3
2.1 Problem Definition	3
2.2 Objectives	3
2.3 Requirements	4
2.4 Users, Constraints and Assumptions	4
CHAPTER 3: LITERATURE REVIEW	6
3.1 The Relational Model and ER Modelling	6
3.2 Normalisation Theory	6
3.3 Transactions and Concurrency	6
3.4 Web Application Security	7
3.5 Automatic Identification in Logistics	7
3.6 Comparative Analysis and Research Gap	7
CHAPTER 4: METHODOLOGY AND SYSTEM DESIGN	9
4.1 Development Methodology	9
4.2 System Architecture	9
4.3 Workflow and Algorithms	10
4.4 Tools and Technologies	11
4.5 Use-Case and Data-Flow Design	12
4.6 Database Design	13
CHAPTER 5: IMPLEMENTATION	17
5.1 Environment and Project Structure	17
5.2 Database Creation	17
5.3 Authentication and Sessions	17
5.4 Booking Module	18
5.5 Public Tracking and QR Module	18
5.6 Administrator Module	19
5.7 Security Implementation	20
CHAPTER 6: RESULTS, TESTING AND DISCUSSION	21
6.1 Functional Testing	21
6.2 Integrity and Concurrency Tests	22
6.3 Performance	22
6.4 Query Outputs	23
6.5 Comparison with Objectives and Discussion	24
CHAPTER 7: CONCLUSION AND FUTURE SCOPE	25
7.1 Summary	25
7.2 Learning Outcomes	25
7.3 Limitations and Future Enhancements	25
7.4 Concluding Remarks	26
REFERENCES	27

LIST OF FIGURES

Figure 4.1: Three-tier architecture of SwiftCourier	9
Figure 4.2: Flowchart of the booking transaction	10
Figure 4.3: Parcel status state-transition diagram	11
Figure 4.4: Use-case diagram	12
Figure 4.5: Data-flow diagram, Level 1	13
Figure 4.6: ER diagram of courier_management (Chen notation)	14
Figure 4.7: Relational schema with keys and foreign-key references	15
Figure 5.1: Booking form and booking confirmation with QR code	18
Figure 5.2: Public tracking page in mobile view, opened by scanning the parcel's QR code	19
Figure 5.3: Administrator dashboard and status update	20
Figure 6.1: Median SQL query and transaction times	23

LIST OF TABLES

Table 2.1: Functional requirements	4
Table 3.1: Comparison of related work and existing approaches	7
Table 4.1: Tools and technologies	11
Table 4.2: Tables, keys and constraints	15
Table 4.3: Functional dependencies and candidate keys	16
Table 5.1: Security measures	20
Table 6.1: Functional test results	21
Table 6.2: Constraint violation tests	22
Table 6.3: SQL query and transaction timings (ms)	22
Table 6.4: Achievement of objectives	24

CHAPTER 1: INTRODUCTION

Courier services move documents and goods every day, and every parcel produces a trail of information: who sent it, who will receive it, what it weighs, what was charged and where it has been. Managing this information reliably is fundamentally a database problem. SwiftCourier designs a relational database for courier booking and tracking and builds a web application on top of it so that customers, staff and the public can use the data directly.

1.1 Background

Relational database management systems store data in tables connected by keys, an approach introduced by Codd [1] and now the basis of almost every business information system. A good relational design stores each fact once, connects tables through primary and foreign keys, and relies on the database engine to enforce the rules that keep data consistent [2], [3].

A courier operation maps naturally onto this model. A customer has an identity and contact details; a parcel is booked by one customer but has its own sender, receiver, type, weight and charge; as it moves it passes through a sequence of events, each with a status, location, time and the staff member who recorded it; and each booking produces a payment. These four kinds of facts are connected by one-to-many relationships, exactly the situation that entity–relationship modelling and normalisation are designed for.

Many small courier counters, however, still keep bookings in registers or spreadsheets. The same parcel may be written down twice with different details, the current status lives only in a phone call, and overwriting a status cell erases the earlier history. Large couriers solve this with proprietary systems that small operators cannot use or study. Yet the tools for a proper system are free: PHP and MariaDB run on any laptop through XAMPP [4], browsers work on laptops and phones alike, and every smartphone camera reads QR codes [5].

1.2 Motivation

- **Visibility:** customers should see where a parcel is without phoning the branch, by tracking ID or by scanning a QR code.
- **Complete history:** every movement should be kept, not overwritten, so that disputes can be answered from the record.
- **Correct billing:** the charge should follow one transparent rule calculated by the system.

- **Integrity:** the database itself should refuse impossible data, such as a parcel for a non-existent customer or a duplicate account.
- **Learning value:** the project exercises every core topic of the course, from ER modelling and normalisation to joins, aggregates, views and transactions, in one working system.

The project also respects a zero-cost constraint: no paid services, cloud databases, payment gateways or external APIs, so that it runs on any laptop and every component can be explained in a viva.

1.3 Scope and Organisation of the Report

SwiftCourier supports customers, administrators (staff) and public visitors. In scope are registration and login, booking with automatic charges, unique tracking IDs, QR codes, simulated payments, public tracking with a timeline, status updates with location and remarks, search and filtering, an SQL-driven dashboard, and deletes that demonstrate referential actions. Real payment gateways, GPS, SMS alerts and route optimisation are future work.

Chapter 2 defines the problem and objectives; Chapter 3 reviews related work; Chapter 4 presents the system and database design; Chapter 5 describes the implementation; Chapter 6 reports the results; and Chapter 7 concludes with future enhancements.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Definition

The problem is to design a relational database, and a web application on top of it, that records courier bookings and the complete movement history of every parcel, keeps the data consistent under all operations, and makes it available to customers, staff and the public.

The difficulty lies in modelling the data correctly. A flat table with one row per booking and columns such as status_1, status_2 and status_3 cannot hold an unknown number of movements, repeats customer details on every row and loses information when a status is overwritten; it suffers from update, insertion and deletion anomalies [2]. Multi-table operations must also be atomic: a booking must create the parcel, its first tracking event and its payment together, and a status change must update the parcel and add a history row together. Finally, tracking IDs must stay unique when customers book at the same moment, and public tracking must show live data without a login and without exposing private details.

2.2 Objectives

1. **Design a normalised relational schema** for users, parcels, tracking history and payments, mapped from an ER diagram and normalised to at least Third Normal Form.
2. **Enforce integrity in the database** with primary keys, foreign keys and referential actions, UNIQUE, NOT NULL, CHECK and ENUM constraints.
3. **Implement transactional booking** that validates input, calculates the charge, generates a unique tracking ID and stores parcel, first event and payment atomically.
4. **Keep an append-only tracking history** in which each status change updates the parcel and inserts a history row in one transaction, following an explicit transition rule.
5. **Provide public tracking and real QR codes** that open live tracking data on any phone.
6. **Build an SQL-driven admin dashboard** with search, filtering and delete operations.
7. **Secure the application** with password hashing, sessions, role checks, prepared statements and CSRF protection, never showing raw errors to users.

2.3 Requirements

Table 2.1: Functional requirements

ID	Requirement	Role
FR1	Register, log in and log out; reject duplicate emails.	Customer, Admin
FR2	Book a parcel with sender, receiver, type, weight, expected date and payment method; charge calculated automatically.	Customer
FR3	Generate a unique tracking ID (CR + YYYYMMDD + 4-digit sequence) and a QR code (view, download, print).	System
FR4	List, search and filter own parcels; view details, payment and history.	Customer
FR5	Track any parcel by ID or QR scan without login, with a timeline from the database.	Public
FR6	View dashboard statistics; search and filter all parcels; view customers with booking counts.	Admin
FR7	Update status, location and remarks, adding a tracking-history row.	Admin
FR8	Delete a parcel (with history and payments); delete a customer only if no parcels exist.	Admin

The main non-functional requirements are **integrity** (rules enforced by the database), **atomicity** (multi-table changes as transactions), **security** (hashed passwords, prepared statements, CSRF tokens, role checks), **usability** (a responsive interface, especially the mobile tracking page), **performance** (pages well within one second) and **portability** (a single SQL file importable through phpMyAdmin on standard XAMPP or WAMP).

2.4 Users, Constraints and Assumptions

There are three roles. **Customers** (logged in) book parcels and view their own parcels and history. **Administrators** (logged in, role admin) view the dashboard, search, update status, view customers and delete. **Public visitors** (no login) track a parcel by ID or QR scan and see only its status and timeline.

- The system runs on a local XAMPP server; phone access for QR scanning assumes the phone and laptop share a Wi-Fi network.
- Payments are simulated: online methods are recorded as Paid with a generated reference, and Cash on Pickup stays Pending until delivery.
- Locations are entered by staff as text; there is no GPS. Bootstrap loads from a CDN, while the QR library is stored locally.

- Each parcel has one booking customer, but the sender may be a different person, so sender details are stored with the parcel.

CHAPTER 3: LITERATURE REVIEW

SwiftCourier rests on established ideas in database theory and practice, together with standard techniques for web security and automatic identification. This chapter reviews the works that directly shaped the design and ends with a comparison that identifies the gap the project fills.

3.1 The Relational Model and ER Modelling

Codd's relational model represents data as relations of tuples identified by keys, and separates the logical structure of data from its storage [1]. Chen's entity-relationship model describes the world as entities, attributes and relationships with defined cardinality and participation, and gives a systematic mapping from an ER diagram to tables [6]. Both remain the foundation of database design teaching [2], [3]. SwiftCourier was designed first as a four-entity ER diagram in Chen notation and then mapped to keyed relations whose relationships are foreign keys.

3.2 Normalisation Theory

Codd introduced the second and third normal forms to remove redundancy caused by partial and transitive dependencies [7], and later, with Boyce, the stronger BCNF, in which every determinant must be a candidate key [8]. Kent summarised the idea as every non-key attribute depending on "the key, the whole key and nothing but the key" [9], and Date discusses deliberate, controlled redundancy for performance [10]. The SwiftCourier schema was derived from an unnormalised register through 1NF, 2NF and 3NF and verified against BCNF, with two summary columns kept deliberately and maintained by transactions.

3.3 Transactions and Concurrency

Haerder and Reuter defined the ACID properties of transactions and how databases recover to a consistent state [11], and Gray and Reuter describe how commit, rollback and locking let many users change shared data safely [12]. InnoDB's locking reads (SELECT ... FOR UPDATE) lock rows until the transaction ends [13], and a UNIQUE constraint guarantees that a duplicate can never be committed [14]. SwiftCourier runs booking and status updates as transactions and generates tracking IDs with a locking read backed by a UNIQUE key.

3.4 Web Application Security

Halfond, Viegas and Orso classified SQL-injection attacks and identified parameterised queries as the most effective defence [15]. OWASP recommends prepared statements for every query [16] and synchroniser tokens against cross-site request forgery [17], and passwords should be stored with a slow salted hash such as bcrypt, available in PHP through password_hash [18]. SwiftCourier applies all three: MySQLi prepared statements [19], per-session CSRF tokens and bcrypt hashing.

3.5 Automatic Identification in Logistics

Logistics systems identify items automatically so that their movement can be recorded without manual typing. QR codes, standardised as ISO/IEC 18004, can encode a URL with error correction and are read by ordinary smartphone cameras [5], while RFID reads tags without line of sight but needs dedicated readers [20]. SwiftCourier uses QR codes because they need no special hardware; the code encodes the public tracking URL and is generated in the browser by an open-source library [21].

3.6 Comparative Analysis and Research Gap

In practice, small courier counters use **paper registers** (cheap but unsearchable and unchecked) or **spreadsheets** (searchable but flat, with repeated customer data and overwritten status), while large couriers use **proprietary platforms** that are complete but closed and costly. Table 3.1 compares these approaches with the foundational work.

Table 3.1: Comparison of related work and existing approaches

Work / approach	Focus	Limitation for this problem	Use in SwiftCourier
Codd [1]; Chen [6]	Relational and ER modelling	Theory and notation only	4-entity ER diagram mapped to keyed tables
Codd [7], [8]; Kent [9]	Normalisation	Full normalisation can cost joins	Schema in BCNF with two controlled summary columns
Haerder & Reuter [11]; Gray & Reuter [12]	Transactions, concurrency	Must be applied by the developer	Atomic booking and status update; locked ID generation
Halfond et al. [15]; OWASP	Web security	Guidance, not a system	Prepared statements, CSRF tokens, bcrypt
ISO/IEC 18004; RFID [20]	Automatic identification	RFID needs special hardware	QR code encoding the tracking URL
Registers and spreadsheets	Small couriers	No integrity; history lost	Replaced by a normalised database

Work / approach	Focus	Limitation for this problem	Use in SwiftCourier
Commercial platforms	Large couriers	Proprietary, costly, closed	Same core capability on free software

The theory is mature and large couriers already offer full tracking, but small operators and students lack an open, low-cost system that applies this theory end to end. SwiftCourier fills this gap by combining a normalised schema, database-enforced integrity, transactional updates, an append-only history and QR-based public tracking on free software.

CHAPTER 4: METHODOLOGY AND SYSTEM DESIGN

4.1 Development Methodology

The project followed a database-first, iterative approach. Requirements were modelled as an ER diagram, mapped to tables and normalised before any application code was written. The SQL script, including sample data and deliberately invalid statements, was tested in phpMyAdmin so the constraints were proven first. The application was then built in increments (authentication, booking, tracking and QR codes, the admin module, deletes), and an automated end-to-end test script was extended and re-run after each increment.

4.2 System Architecture

SwiftCourier uses a three-tier architecture (Figure 4.1). The client tier is any browser on a laptop or phone; the application tier is PHP served by Apache, with shared code in `includes/` and settings in `config/`; and the data tier is the courier_management database in MariaDB, accessed only through MySQLi prepared statements. QR codes are rendered in the browser by a local JavaScript library.

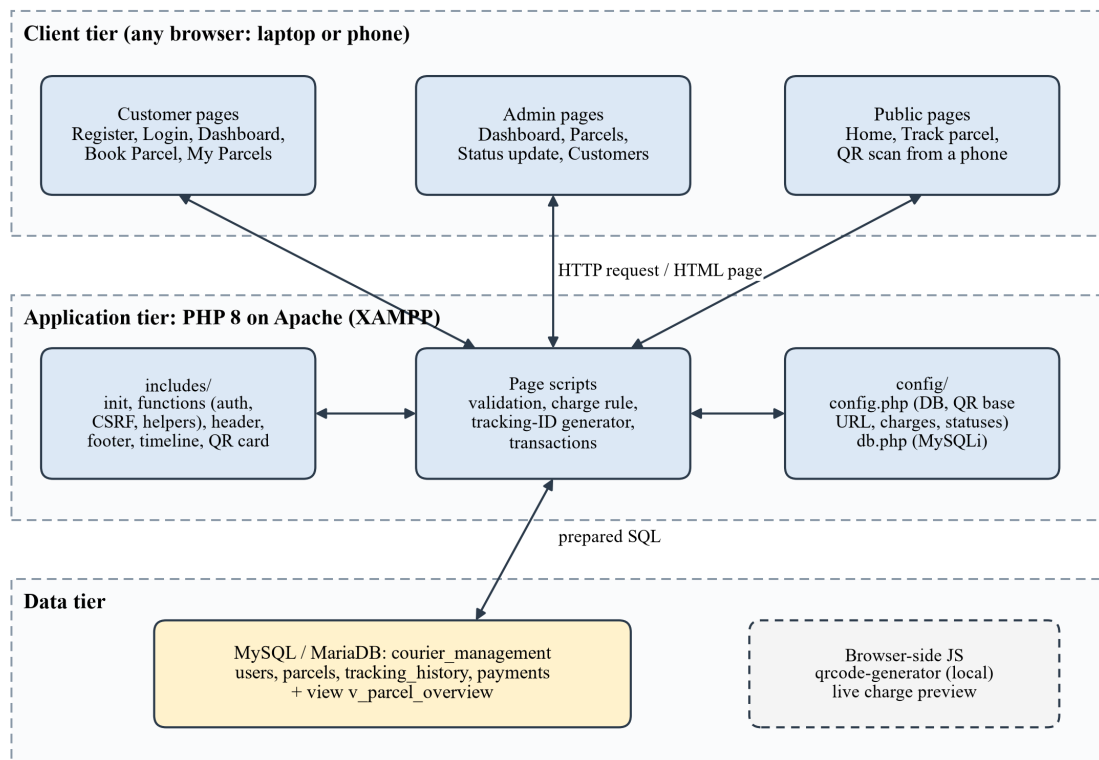


Figure 4.1: Three-tier architecture of SwiftCourier

4.3 Workflow and Algorithms

A customer books a parcel; the database stores the booking and its first event; the system issues a tracking ID and QR code; an administrator updates the status as the parcel moves, each update adding a history row; and anyone with the tracking ID or QR code sees the new status immediately. The booking transaction is shown in Figure 4.2. The status update follows the same pattern: the parcel row is locked with `SELECT ... FOR UPDATE`, the request is refused if the parcel is final or the transition is not allowed, and otherwise `current_status` is updated and a history row inserted before `COMMIT`.

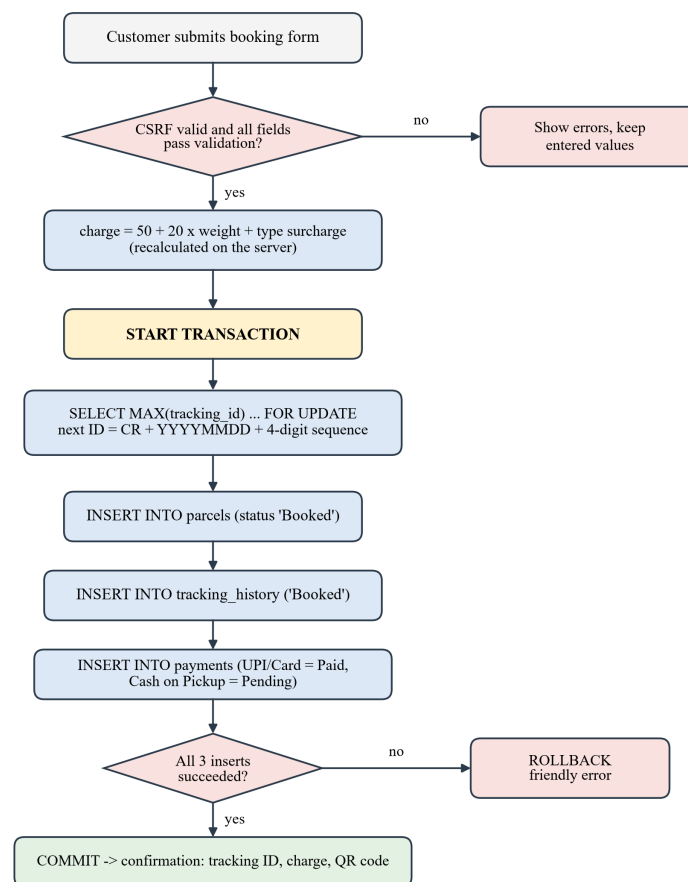


Figure 4.2: Flowchart of the booking transaction

Delivery charge. $\text{charge} = ₹50 \text{ base} + ₹20 \times \text{weight (kg)} + \text{type surcharge}$ (Electronics ₹30, Fragile ₹40, others ₹0). The rule is defined once in `config.php`; the browser shows a preview, but the server always recalculates. A 2.5 kg Fragile parcel costs $50 + 50 + 40 = ₹140$.

Tracking ID. Each ID is CR + YYYYMMDD + a four-digit daily sequence (for example CR202610050001), generated inside the booking transaction:

```
prefix = "CR" + today (YYYYMMDD)
last   = SELECT MAX(tracking_id) FROM parcels
        WHERE tracking_id LIKE prefix + '%' FOR UPDATE    -- locks the rows read
seq    = (last is NULL) ? 1 : last 4 digits of last + 1
id     = prefix + zero_pad(seq, 4)                       -- INSERT protected by UNIQUE(tracking_id)
```

Status transitions. Status is a finite state machine (Figure 4.3) defined once in config.php: Booked → Picked Up → In Transit → Out for Delivery → Delivered; In Transit may repeat for another hub; any active state may become Failed; a Failed parcel may be re-attempted or Returned; Delivered and Returned are final. The admin menu lists only valid next states, and forged requests are rejected on the server.

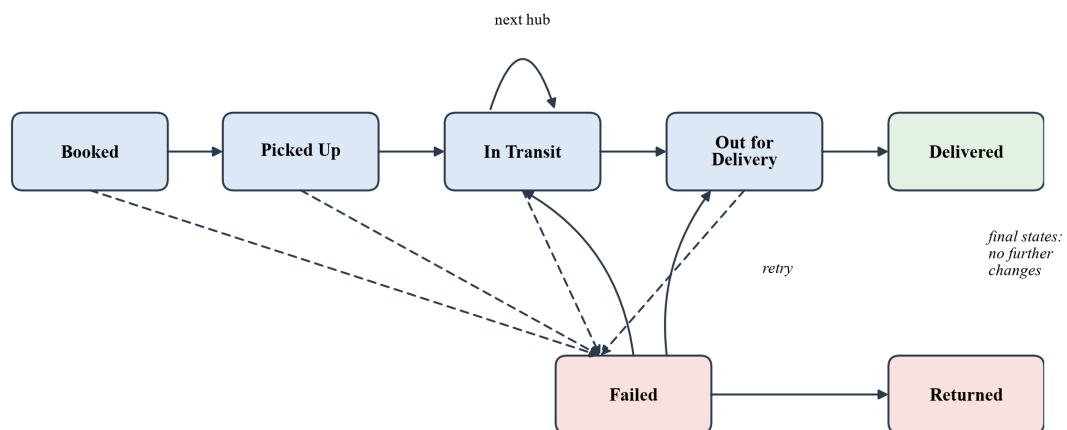


Figure 4.3: Parcel status state-transition diagram

QR tracking URL. The QR code encodes base_url/track.php?tracking_id=.... The base URL is configurable; when the site is opened as localhost, the application substitutes the laptop’s LAN IP address so that a phone on the same Wi-Fi can open the link.

4.4 Tools and Technologies

Table 4.1: Tools and technologies

Layer	Technology	Purpose
Database	MariaDB 10.4 (MySQL-compatible), InnoDB	Tables, constraints, transactions, view
Server and back end	Apache 2.4 and PHP 8.2 (MySQLi) on XAMPP [4]	Pages, validation, sessions, prepared statements
Front end	HTML5, CSS3, Bootstrap 5.3 [22], vanilla JavaScript	Responsive UI, charge preview, QR rendering

Layer	Technology	Purpose
QR codes	qrcode-generator 1.4.4 (MIT) [21]	QR generation in the browser
Tools	phpMyAdmin; Bash + cURL, headless Chrome, PHP CLI	SQL import and queries; testing and timing

4.5 Use-Case and Data-Flow Design

Figure 4.4 shows the use cases of the three actors, and Figure 4.5 the Level 1 data-flow diagram, whose four data stores correspond one-to-one with the database tables. At context level the system exchanges booking details, tracking IDs and parcel lists with customers; tracking IDs and timelines with public visitors; and status updates and statistics with administrators.

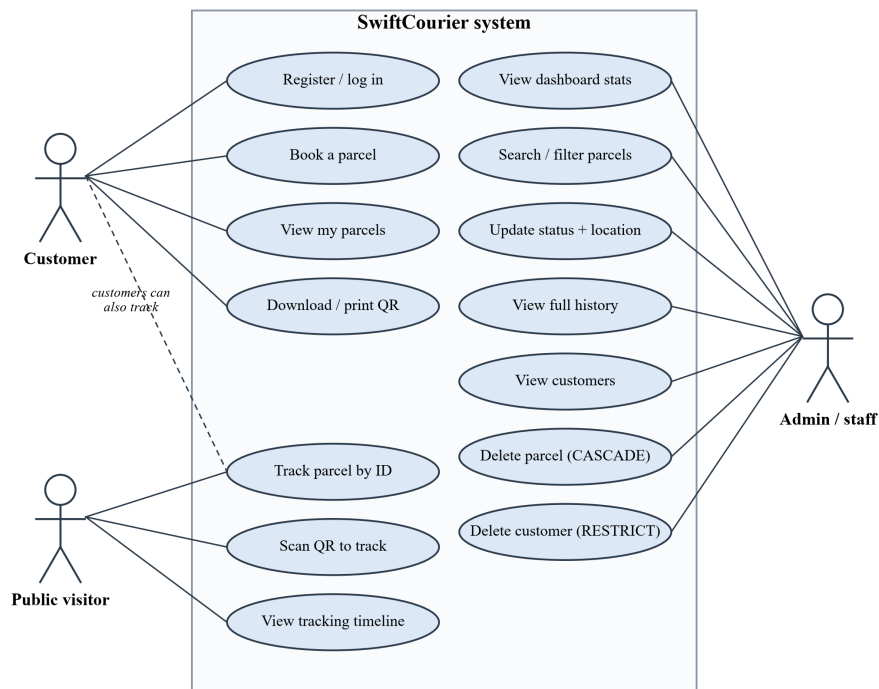


Figure 4.4: Use-case diagram

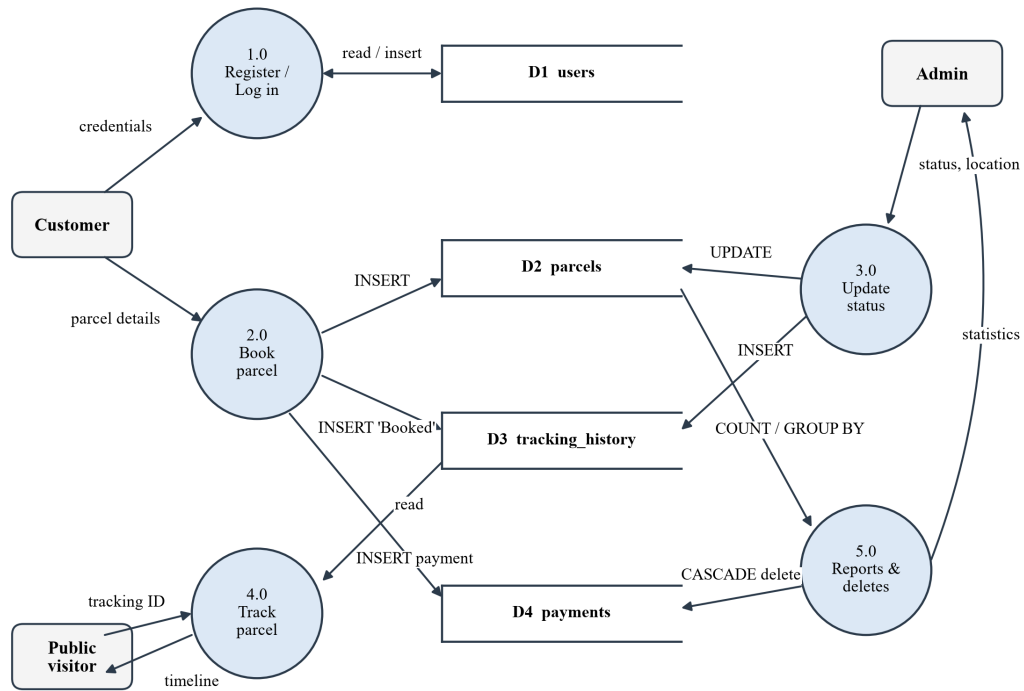
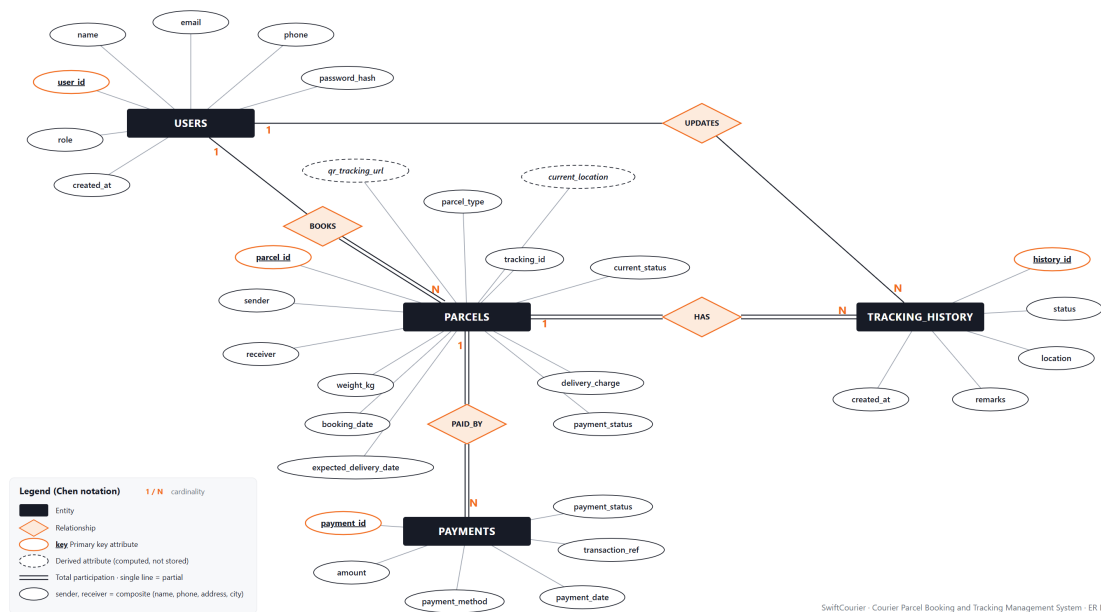


Figure 4.5: Data-flow diagram, Level 1

4.6 Database Design

4.6.1 ER Diagram

The ER diagram (Figure 4.6) has four entities and four one-to-many relationships. A **USER BOOKS** many **PARCELS** (parcel participation total); each **PARCEL HAS** many **TRACKING_HISTORY** events (total on both sides, since every booking inserts a 'Booked' event) and is **PAID_BY** one or more **PAYMENTS** (allowing later refunds or retries); and a **USER UPDATES** many events (partial, since updated_by may be NULL). Sender and receiver are composite attributes; current_location and qr_tracking_url are derived and not stored.



SwiftCourier - Courier Parcel Booking and Tracking Management System - ER Diagram

Figure 4.6: ER diagram of courier_management (Chen notation)

4.6.2 Relational Schema and Constraints

Each entity becomes a table with a surrogate integer primary key, each relationship a foreign key on the many side, composite attributes become atomic columns, and derived attributes are not stored (Figure 4.7). Business keys that people see, tracking_id, email and transaction_ref, are UNIQUE. The view v_parcel_overview joins parcels with users and selects each parcel's latest location with a correlated subquery. Table 4.2 summarises the tables and their constraints.

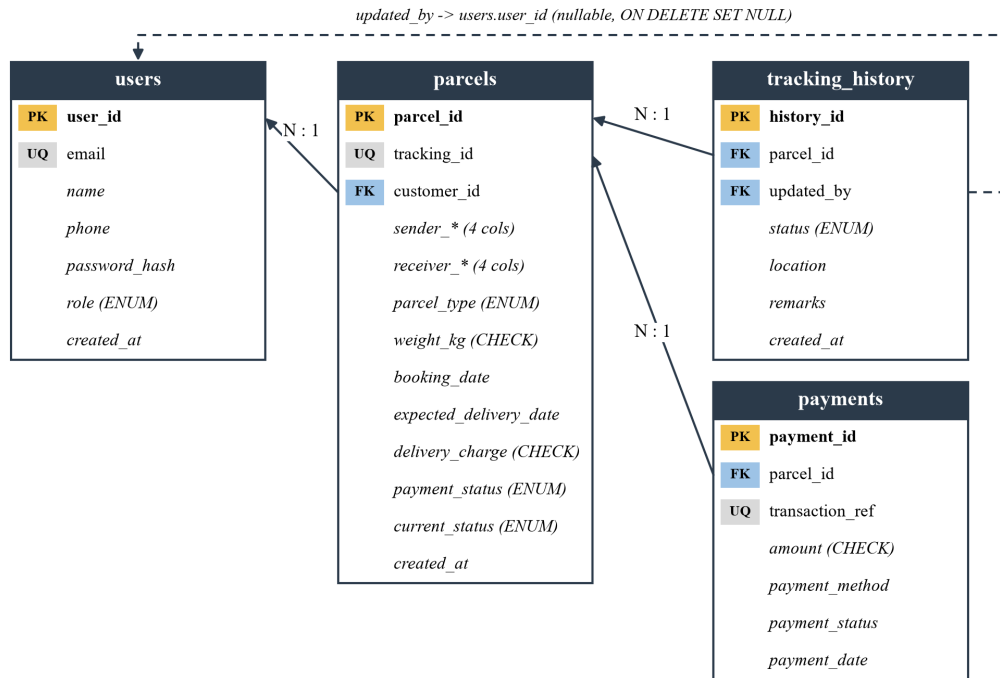


Figure 4.7: Relational schema with keys and foreign-key references

Table 4.2: Tables, keys and constraints

Table	Key columns	Other constraints
users	PK user_id; UNIQUE email	name, phone, password_hash NOT NULL; role ENUM('customer','admin')
parcels	PK parcel_id; UNIQUE tracking_id; FK customer_id → users (ON DELETE RESTRICT)	parcel_type, payment_status, current_status ENUMs; CHECK 0 < weight_kg ≤ 50; CHECK delivery_charge ≥ 0
tracking_history	PK history_id; FK parcel_id → parcels (CASCADE); FK updated_by → users (SET NULL)	status ENUM (7 values); location NOT NULL; index (parcel_id, created_at)
payments	PK payment_id; FK parcel_id → parcels (CASCADE); UNIQUE transaction_ref	payment_method, payment_status ENUMs; CHECK amount ≥ 0

4.6.3 Normalisation

Normalisation started from the relation a paper register produces: BOOKING(tracking_id, customer_name, customer_email, customer_phone, sender..., receiver..., weight, charge, status_1, location_1, status_2, location_2, ..., pay_mode, txn_ref).

- **1NF:** status_1, location_1, status_2, ... is a repeating group of unknown size, so each event becomes a row of TRACKING_HISTORY and every column holds a

single value.

- **2NF:** with events keyed by (tracking_id, time), sender, receiver and weight would depend on tracking_id alone (a partial dependency), so they stay in PARCELS and each event gets its own key, history_id.
- **3NF:** tracking_id → customer_email → customer_name, customer_phone is transitive, so customer facts move to USERS; transaction_ref → amount, method moves payment facts to PAYMENTS; derived values are not stored.
- **BCNF:** the remaining dependencies (Table 4.3) all have a candidate key as determinant, so every table is in BCNF [8].

Table 4.3: Functional dependencies and candidate keys

Table	Functional dependencies	Candidate keys	Form
users	user_id → all; email → all	user_id, email	BCNF
parcels	parcel_id → all; tracking_id → all	parcel_id, tracking_id	BCNF
tracking_history	history_id → all	history_id	BCNF
payments	payment_id → all; transaction_ref → all	payment_id, transaction_ref	BCNF

Controlled redundancy. parcels.current_status equals the latest tracking_history status, and parcels.payment_status summarises the payments rows. They are kept because dashboards and filters group by them constantly, and they are only ever changed in the same transaction as the source row, so they cannot disagree with it [10].

CHAPTER 5: IMPLEMENTATION

5.1 Environment and Project Structure

The system was developed on Windows 11 with XAMPP (Apache 2.4, PHP 8.2.12, MariaDB 10.4.32) and runs at <http://localhost/courier/>. The structure is deliberately flat: customer pages (dashboard, book_parcel, my_parcel, parcel_details), public pages (index, track, qr), authentication (login, register, logout), an admin/ folder (index, parcels, parcel, customers, delete), config/ (settings and the MySQLi connection), includes/ (initialisation, helper and authentication functions, header, footer, timeline and QR components), assets/ (CSS, JavaScript and the QR library) and database/ (the schema script and demonstration queries).

5.2 Database Creation

A single script, `courier_management.sql`, imported through phpMyAdmin, recreates the database with its four tables, constraints and view, and inserts sample data: 6 users (2 admins, 4 customers), 12 parcels covering every status, 44 tracking-history rows and 12 payments, with demonstration passwords stored as bcrypt hashes. The parcels table illustrates the constraints:

```
CREATE TABLE parcels (
  parcel_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  tracking_id    CHAR(14) NOT NULL,
  customer_id    INT UNSIGNED NOT NULL,
  ...           -- sender_* and receiver_* columns
  parcel_type    ENUM('Document','Package','Electronics','Fragile','Other') NOT NULL,
  weight_kg      DECIMAL(6,2) NOT NULL,
  delivery_charge DECIMAL(10,2) NOT NULL,
  current_status ENUM('Booked', ... , 'Returned') NOT NULL DEFAULT 'Booked',
  CONSTRAINT uq_parcel_tracking UNIQUE (tracking_id),
  CONSTRAINT fk_parcel_customer FOREIGN KEY (customer_id)
    REFERENCES users(user_id) ON DELETE RESTRICT,
  CONSTRAINT chk_parcel_weight CHECK (weight_kg > 0 AND weight_kg <= 50)
) ENGINE=InnoDB;
```

5.3 Authentication and Sessions

Every page includes `init.php`, which starts the session with `HttpOnly` and `SameSite` cookies, installs an exception handler that logs errors and shows a friendly message, and opens the MySQLi connection in exception mode with strict SQL mode enabled. Registration validates input, rejects existing emails (also catching duplicate-key error 1062), hashes the password with `password_hash` and inserts the user. Login verifies the password with `password_verify` and regenerates the session ID to prevent session fixation. The helpers `require_login`, `require_customer` and `require_admin` protect every page by role, and customers can open only parcels whose `customer_id` matches their session.

The booking form collects sender and receiver details, parcel type, weight, expected date and payment method. The browser validates fields and previews the charge, and the server repeats every check before running one transaction:

```
$conn->begin_transaction();
try {
    $trackingId = generate_tracking_id($conn);           // locking read
    INSERT INTO parcels (...) VALUES (...);              // status 'Booked'
    INSERT INTO tracking_history (parcel_id, status, location, ...)
        VALUES (?, 'Booked', ?, ...);
    INSERT INTO payments (parcel_id, amount, payment_method, ...)
        VALUES (?, ?, ?, ...);                        // all prepared statements
    $conn->commit();
} catch (mysqli_sql_exception $ex) {
    $conn->rollback();                                  // nothing is saved
}
```

On success the customer sees a confirmation with the tracking ID, status, charge, payment record, history and QR code (Figure 5.1, right; the booking form is on the left). The customer dashboard summarises bookings with COUNT and conditional SUM, and My Parcels supports LIKE search and a status filter.

The image displays two views of the Swift Courier web application. The left view is the 'Book a Parcel' form, which includes sections for 'Sender details', 'Receiver details', and 'Parcel details'. The 'Sender details' section has fields for name, phone, and address. The 'Receiver details' section has similar fields. The 'Parcel details' section includes 'Parcel type' (a dropdown menu) and 'Weight (kg)' (a numeric input). The right view shows the 'Booking confirmed!' status for tracking ID CR20261006001. It features a QR code, a 'Public tracking page' link, and a table of charges. The charges table lists 'Base charge', '13 kg + ₹20', and 'Type surcharge', with a total of ₹140.00. Below the charges table, there is a section for 'Payment method' and a 'Confirm Booking' button.

Figure 5.1: Booking form and booking confirmation with QR code

track.php needs no login. It validates the tracking-ID format, reads the parcel and its history with prepared statements, and shows the tracking ID, sender, receiver, type, weight, dates, current status, origin, destination, current location and delivery information, followed by a progress bar and a timeline built from the `tracking_history` rows. Phone numbers and addresses are not shown publicly. The page is mobile-first because it is what a phone opens after a QR scan (Figure 5.2). The QR code is drawn on a canvas by `qrcode-generator` with medium error correction, and can be downloaded as PNG or printed as a label from `qr.php`.

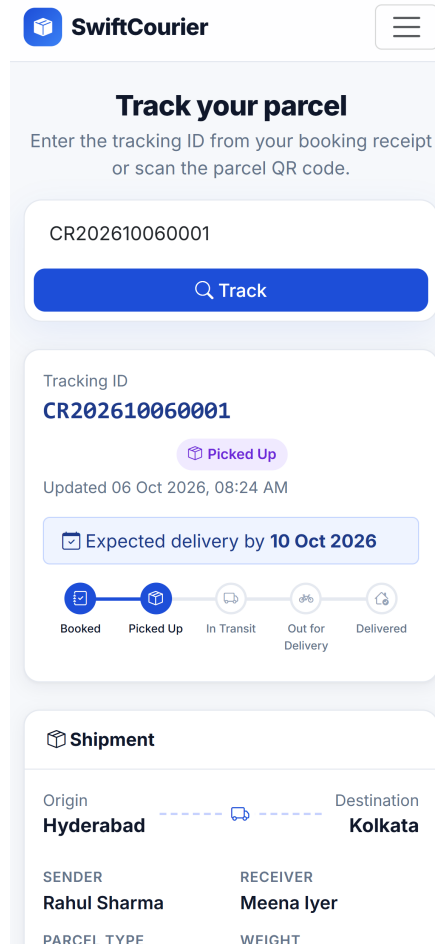


Figure 5.2: Public tracking page in mobile view, opened by scanning the parcel's QR code

5.6 Administrator Module

Dashboard. All figures come from SQL: a COUNT of customers, a GROUP BY on current_status for the status cards, a SUM of paid payments for revenue, a JOIN for recent bookings, a JOIN with GROUP BY and HAVING for top customers, and a three-table join for the latest updates. A collapsible panel displays these queries (Figure 5.3, left; the status-update page is on the right).

Search and customers. The parcels page builds its WHERE clause from optional, always-bound conditions (LIKE across tracking ID, names, emails and phones; status; customer) and paginates with LIMIT/OFFSET. The customers page uses LEFT JOIN with GROUP BY so customers with no bookings still appear.

Status update. The parcel page shows full details, payments and history with the name and role of each updater. The update transaction locks the row, checks the final state and allowed transitions, updates current_status, inserts the history row and, when a Cash on Pickup parcel is delivered, marks its payment Paid.

Deletes. Deleting a parcel issues one DELETE; ON DELETE CASCADE removes its history and payments, and the message reports how many rows went. Deleting a customer who still has parcels is refused by MySQL with error 1451 (ON DELETE RESTRICT), which the application explains. Both require POST, a CSRF token and confirmation.

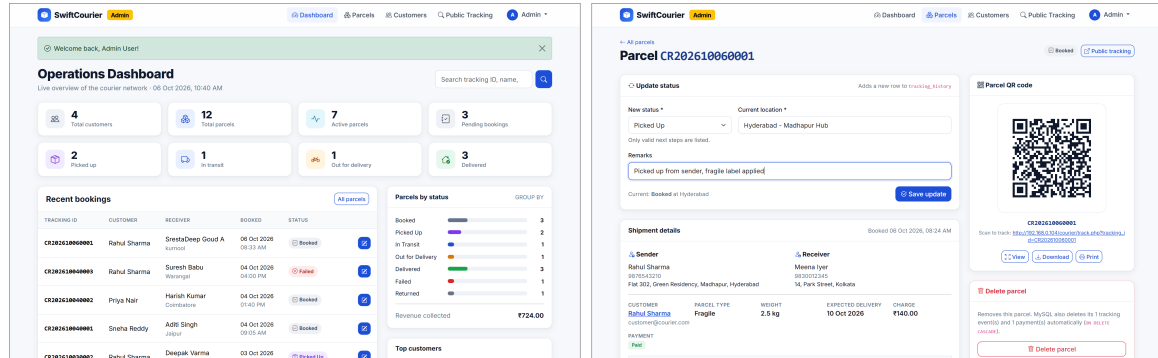


Figure 5.3: Administrator dashboard and status update

5.7 Security Implementation

Table 5.1: Security measures

Threat	Measure
SQL injection	MySQLi prepared statements with bound parameters for every query
Password theft; session fixation	bcrypt via password_hash / password_verify; session ID regenerated on login; HttpOnly, SameSite cookies
Cross-site request forgery	Per-session token in every form and in the logout link
Unauthorised access	Role checks on every page; ownership check on customer parcels
Cross-site scripting; information leakage	All output escaped with htmlspecialchars; errors logged, not shown; .htaccess blocks config, includes and database folders
Invalid data	Server-side validation plus database constraints and strict SQL mode

CHAPTER 6: RESULTS, TESTING AND DISCUSSION

6.1 Functional Testing

Functional testing was automated with a Bash script that drives the running application over HTTP with cURL, using separate customer and administrator sessions and extracting CSRF tokens from the pages. The database was reset before the run. The script made 37 content checks and 7 access-control checks, and all 44 passed (Table 6.1).

Table 6.1: Functional test results

#	Test case	Expected result	Result
1	Public tracking with valid, malformed and unknown IDs	Correct timeline; validation message; “No parcel found”	Pass
2	Register; duplicate email; forged CSRF token	Account created; “already exists”; rejected	Pass
3	Wrong then correct password	Error, then dashboard	Pass
4	Customer opens another customer’s parcel	Not found	Pass
5	Book Fragile 2.5 kg parcel; second booking same day	IDs ...0001 and ...0002; charge ₹140	Pass
6	Booking with invalid data	Errors listed; nothing inserted	Pass
7	Admin search, status filter, customers page	Correct results; new parcel visible	Pass
8	Status update; public page after update	New history row; new status shown immediately	Pass
9	Invalid status; Booked → Delivered; update a Delivered parcel	All rejected with a reason	Pass
10	Deliver a Cash on Pickup parcel	Payment changes Pending → Paid	Pass
11	Delete customer with parcels; without parcels; delete parcel	Refused (1451); deleted; history and payment cascaded	Pass
12	Guest and customer access to protected pages; logout	Redirected; session ended	Pass

QR scanning on a real device. The QR code was scanned with a team member’s smartphone connected to the same Wi-Fi network as the laptop. It opened the public tracking page for the parcel and displayed its current status and timeline read live from MySQL. The generated codes were also decoded with an independent decoder (OpenCV), which returned exactly the parcel’s tracking URL.

6.2 Integrity and Concurrency Tests

To confirm that integrity is enforced by the database and not only by PHP, six invalid statements were run directly in MariaDB, and each was rejected (Table 6.2). MariaDB reports a failed CHECK as error 4025, while MySQL 8 reports the same rule as 3819 [23].

Table 6.2: Constraint violation tests

Attempted statement	Constraint	Result
Parcel weighing 75 kg	CHECK chk_parcel_weight	ERROR 4025
Parcel for customer_id 99 (does not exist)	FOREIGN KEY	ERROR 1452
Second user with customer@courier.com	UNIQUE email	ERROR 1062
Parcel with existing tracking ID	UNIQUE tracking_id	ERROR 1062
Delete user who still has parcels	ON DELETE RESTRICT	ERROR 1451
Set current_status = 'Lost' (strict mode)	ENUM domain	ERROR 1265

Without strict mode, XAMPP's MariaDB accepted the invalid ENUM value with only a warning, so the application now enables strict mode on its own connection. For concurrency, 10 PHP processes each made 5 bookings at the same time using the application's transactional logic: all 50 rows were inserted with 50 distinct tracking IDs, from CR202610060001 to CR202610060050 with no gaps, and no transaction failed. The locking read serialised ID generation, so the UNIQUE constraint was never needed as a last resort.

6.3 Performance

Each query was timed 300 times (transactions 200 times, rolled back) with a high-resolution timer on the development laptop (Table 6.3, Figure 6.1). Page response times were measured with cURL over 40 requests per page.

Table 6.3: SQL query and transaction timings (ms)

Query	Average	Median	95th pct.
Total customers (COUNT)	0.227	0.215	0.345
Parcels by status (GROUP BY)	0.214	0.200	0.352
Recent bookings (JOIN, ORDER BY)	0.316	0.281	0.510
Bookings per customer (LEFT JOIN, GROUP BY)	0.498	0.494	0.708
Tracking history (3-table JOIN)	0.402	0.345	0.652

Query	Average	Median	95th pct.
Parcel overview (VIEW with subquery)	0.804	0.711	1.456
Booking transaction (3 INSERTs)	2.772	2.523	4.037
Status update transaction (UPDATE + INSERT)	3.085	2.662	5.170

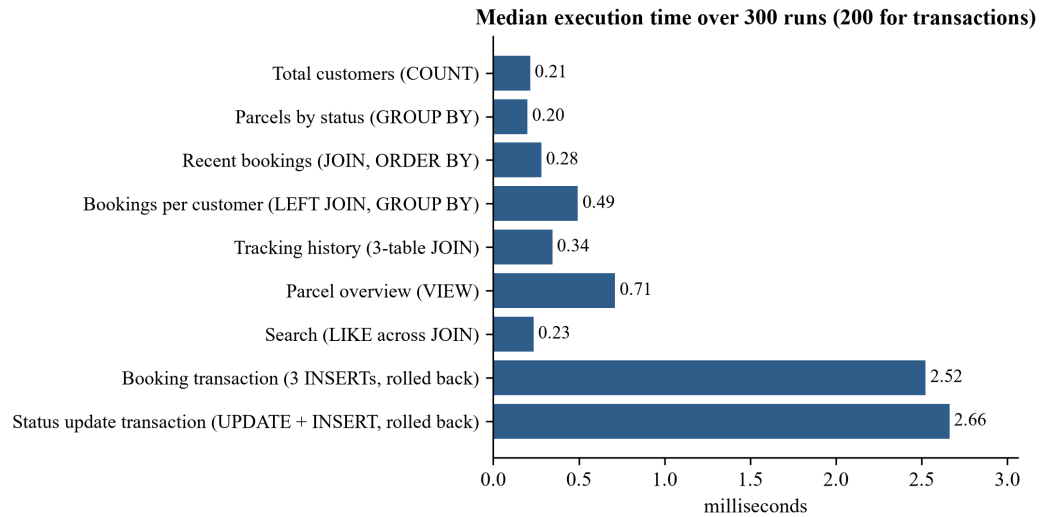


Figure 6.1: Median SQL query and transaction times

Every query completed in well under 1 ms at the median and both transactions in under 3 ms. The view was the slowest query because its correlated subquery runs once per parcel, which is why dashboards use the `current_status` summary column instead. Pages were served with a median of 31–44 ms; the admin dashboard, which runs six queries, was slowest (median 43.5 ms, 95th percentile 156 ms). These measurements used sample data and one user, so they show that the design adds no unnecessary overhead rather than predicting large-scale performance.

6.4 Query Outputs

The demonstration queries return the answers the project set out to provide. On the sample data, parcels by status were Booked 2, Picked Up 2, In Transit 2, Out for Delivery 1, Delivered 3, Failed 1 and Returned 1. Bookings per customer (LEFT JOIN with GROUP BY) were Rahul Sharma 5 (₹500), Priya Nair 3 (₹200), Arjun Mehta 2 (₹300) and Sneha Reddy 2 (₹204). Revenue from paid payments was ₹780 (UPI ₹300, Card ₹330, Net Banking ₹150). The three-table join for CR202610010001 returned five events, from Booked in Hyderabad to Out for Delivery in Chennai, which are the same rows the tracking page and the QR scan display.

6.5 Comparison with Objectives and Discussion

Table 6.4: Achievement of objectives

Objective	Evidence	Status
1. Normalised schema	ER diagram; FD analysis (Table 4.3): BCNF	Achieved
2. Database-enforced integrity	Table 4.2; all six violations rejected (Table 6.2)	Achieved
3. Transactional booking	Tests 5–6; 50/50 unique IDs under concurrency	Achieved
4. Append-only history	Tests 8–10; transitions enforced, rows kept	Achieved
5. Public tracking and QR	Test 1; QR scanned on a smartphone; decoded URL verified	Achieved
6. SQL dashboard, search, deletes	Tests 7 and 11; dashboard SQL panel	Achieved
7. Security	Tests 2–4 and 12; Table 5.1	Achieved

The results confirm the central idea of the project: once the data is modelled correctly and the rules live in the database, the application becomes simpler and more reliable. Invalid states, such as a parcel without a customer, a duplicate tracking ID or orphaned history, cannot arise even if a page contains a bug. The append-only history proved its value: the timeline, the “updated by” information, the latest location and the delivery date all come from one table without extra columns. Testing also showed that constraints should be verified on the actual server, since XAMPP’s default non-strict mode silently altered an invalid ENUM value until strict mode was enabled.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Summary

SwiftCourier is a courier booking and tracking system built on a normalised relational database. An ER diagram was mapped to four tables and one view, analysed to BCNF, and protected by keys, referential actions and value constraints. A PHP application on XAMPP lets customers book parcels, generates unique tracking IDs inside transactions, produces QR codes that open a public tracking page, lets administrators update status through an explicit state machine with an append-only history, and computes all statistics with SQL. All 44 checks passed, every invalid statement was rejected, 50 concurrent bookings received unique consecutive IDs, and the QR code worked on a real smartphone.

7.2 Learning Outcomes

- Modelling a real problem as an ER diagram and mapping it to a normalised schema justified by functional dependencies.
- Choosing referential actions (CASCADE, RESTRICT, SET NULL) from the meaning of each relationship.
- Using transactions and locking reads for atomic and concurrent-safe updates.
- Writing joins, outer joins, aggregates, GROUP BY, HAVING, views and subqueries for real reports.
- Building and testing a secure web application around a database.

7.3 Limitations and Future Enhancements

Payments are simulated, locations are typed by staff, the system runs on a local server, there is a single admin role, and performance was measured with sample data only. Future enhancements are:

1. **Branches and staff roles:** branch, hub and delivery-agent tables, with each event linked to a branch and role-based permissions.
2. **Real payment gateway:** UPI and card payments stored in the existing payments table, which already allows several rows per parcel.
3. **Notifications:** SMS and email alerts to sender and receiver at each status change.
4. **Proof of delivery:** a photo or OTP captured in a delivery-agent app and stored with the Delivered event.
5. **Live location:** GPS coordinates on events, a route map and arrival estimates.

6. **Online deployment:** hosting with HTTPS so QR codes work on any network, plus monthly revenue and delivery-time reports.

7.4 Concluding Remarks

SwiftCourier shows that the core of a professional courier tracking service, one consistent record of every parcel and its complete journey available to anyone with a tracking number, can be built from first principles of database design on free software. Keeping the rules in the database, using transactions for every multi-table change and preserving history instead of overwriting it make the system correct, explainable and easy to extend.

REFERENCES

- [1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.
- [2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.
- [3] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [4] Apache Friends, "XAMPP Apache + MariaDB + PHP + Perl." [Online]. Available: <https://www.apachefriends.org/>
- [5] Information Technology—Automatic Identification and Data Capture Techniques—QR Code Bar Code Symbology Specification, ISO/IEC Standard 18004:2015, International Organization for Standardization, Geneva, Switzerland, 2015.
- [6] P. P.-S. Chen, "The entity-relationship model—Toward a unified view of data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.
- [7] E. F. Codd, "Further normalization of the data base relational model," in *Data Base Systems*, R. Rustin, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1972, pp. 33–64.
- [8] E. F. Codd, "Recent investigations in relational data base systems," in *Proc. IFIP Congress 74*, Stockholm, Sweden, 1974, pp. 1017–1021.
- [9] W. Kent, "A simple guide to five normal forms in relational database theory," *Communications of the ACM*, vol. 26, no. 2, pp. 120–125, Feb. 1983, doi: 10.1145/358024.358054.
- [10] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.
- [11] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, Dec. 1983, doi: 10.1145/289.291.
- [12] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Mateo, CA, USA: Morgan Kaufmann, 1993.
- [13] Oracle Corporation, "Locking reads," *MySQL 8.0 Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html>
- [14] Oracle Corporation, "FOREIGN KEY constraints," *MySQL 8.0 Reference Manual*. [Online]. Available:

<https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>

- [15] W. G. J. Halfond, J. Viegas, and A. Orso, "A classification of SQL-injection attacks and countermeasures," in *Proc. IEEE Int. Symp. Secure Software Engineering (ISSSE)*, Arlington, VA, USA, 2006.
- [16] OWASP Foundation, "SQL Injection Prevention Cheat Sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [17] OWASP Foundation, "Cross-Site Request Forgery Prevention Cheat Sheet," *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- [18] The PHP Group, "password_hash," *PHP Manual*. [Online]. Available: <https://www.php.net/manual/en/function.password-hash.php>
- [19] The PHP Group, "Prepared statements," *PHP Manual: MySQL Improved Extension*. [Online]. Available: <https://www.php.net/manual/en/mysqli.quickstart.prepared-statements.php>
- [20] K. Finkenzeller, *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards, Radio Frequency Identification and Near-Field Communication*, 3rd ed. Chichester, U.K.: Wiley, 2010.
- [21] K. Arase, "qrcode-generator: QR code generator implementation in JavaScript and other languages," GitHub repository. [Online]. Available: <https://github.com/kazuhikoarase/qrcode-generator>
- [22] Bootstrap Team, "Bootstrap 5.3 documentation." [Online]. Available: <https://getbootstrap.com/docs/5.3/>
- [23] MariaDB Foundation, "CONSTRAINT," *MariaDB Knowledge Base*. [Online]. Available: <https://mariadb.com/kb/en/constraint/>